

Федеральное государственное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа № 5

Вариант 6507

Выполнил: Зыков Андрей Алексеевич

Группа: Р3106

Проверила: Ткешелашвили Н.М.,

Преподаватель практики факультета ПИиКТ

Санкт-Петербург 2025

Задание

По выданному преподавателем варианту разработать программу асинхронного обмена данными с внешним устройством. При помощи программы осуществить ввод или вывод информации, используя в качестве подтверждения данных сигнал (кнопку) готовности ВУ.

1. Программа осуществляет асинхронный вывод данных на ВУ-1
2. Программа начинается с адреса 39E₁₆. Размещаемая строка находится по адресу 5C0₁₆.
3. Строка должна быть представлена в кодировке ISO-8859-5.
4. Формат представления строки в памяти: АДР1: СИМВ2 СИМВ1 АДР2: СИМВ4 СИМВ3 ... СТОП_СИМВ.
5. Ввод или вывод строки должен быть завершён по символу с кодом 00 (NUL). Стоп символ является обычным символом строки и подчиняется тем же правилам расположения в памяти что и другие символы строки.

Основная часть

ORG 0x39E ;Начало программы

WORD_BEG: WORD 0x5C0 ;Начало слова (адрес)

STOP_SYM: WORD 0x00 ;Стоп символ

CURR: WORD 0x5C0 ;Текущая ячейка (адрес)

SAVE: WORD ? ;Ячейка сохранения

CUT: WORD 0x00FF ;Значение для среза символа (15...8)

START: CLA

S1: IN 3 ;Чтение регистра состояния ВУ

AND #0x40 ;Бит 6 SR == 0 ("Готов" нажата?)

BEQ S1 ;Нет - "спин-луп"

LD (CURR)+ ;Загрузка 2-х символов

ST SAVE ;Сохранение фрагмента

AND CUT ;Срезать i+1 символ: 0x00S1S1

BEQ stop ;Если стоп символ, переход к завершению

OUT 2 ;Вывести i символ

S2: IN 3 ;Чтение регистра состояния ВУ

AND #0x40 ;Бит 6 SR == 0 ("Готов" нажата?)

BEQ S2 ;Нет - "спин-луп"

LD SAVE ;Загрузка фрагмента

SWAB ;Поменять символы местами (i+1, i) -> (i, i+1)

AND CUT ;Срезать символ i: 0x00S2S2

BEQ stop ;Если стоп символ, переход к завершению

OUT 2 ;Вывести i+1 символ

JUMP START ;Возврат в начало

stop: OUT 2

HLT ;Останов

ORG 0x5C0 ;Начало слова

WORD 0xC1CF ;Символы 2, 1 (С, Я)

WORD 0xC0C2 ;Символы 4, 3 (Р, Т)

WORD 0xB1B5 ; Символы 6, 5 (Б, Е)

WORD 0x0000 ; Символ 7 (СТОП)

Адрес	Код команды	Мнемоника	Описание
39E	05C0	WORD_BEG	Адрес начала слова
39F	0000	STOP_SYM	Символ конца слова
3A0	05C0	CURR	Текущая ячейка с символами
3A1	0000	SAVE	Ячейка сохранения
3A2	00FF	CUT	Строка для среза первых 8-ми бит
3A3	0200	CLA	0 -> AC
3A4	1203	IN 3	Ожидание готовности ВУ-1
3A5	2F40	AND #40	
3A6	F0FD	BEQ IP-3	
3A7	AAF8	LD (IP-8)+	Косв. автоинкрементная адр. M(5C0) -> AC M(3A0) + 1 -> M(3A0)
3A8	EEF8	ST IP-8	Пр. отн. адр. AC -> M(3A1) - сохранение 2-х символов
3A9	2EF8	AND IP-8	Пр. отн. адр. M(3A2) & AC -> AC - срез 1-ого символа
3AA	F010	BEQ IP+10	Переход, если Z==1 IP+10 -> IP
3AB	1302	OUT 2	Запись в регистр ВУ-1
3AC	1203	IN 3	Ожидание готовности ВУ-1
3AD	2F40	AND #40	
3AE	F0FD	BEQ IP-3	

3AF	AEF1	LD IP-15	Пр. отн. адр. M(3A1) -> AC - загрузка сохраненных символов
3B0	0680	SWAB	AC7...AC0 <-> AC15...AC8 - поменять символы местами
3B1	2EF0	AND IP-16	Пр. отн. адр. M(3A2) & AC -> AC - срез 1-ого символа
3B2	F002	BEQ IP+2	Переход, если Z==1 IP+2 -> IP
3B3	1302	OUT 2	Запись в регистр ВУ-1
3B4	CEEE	JUMP IP-18	IP-18 -> IP
3B5			
3B6	0100	HLT	Останов
5C0	BEB1		СИМВ2 СИМВ1
5C1	00B1		СТОП_СИМВ СИМВ3

Область представления

CURR, WORD_BEG - 11-разрядные ссылки на ячейку

STOP_SYM - 8-разрядное число, хранящее кодировку стоп символа

SAVE - 16-разрядное число, хранящее 2 закодированных символа

CUT - 16-разрядное число хранящее маску для среза 1-ого символа

5C0, ... - 16-разрядные ячейки, хранящие 2 закодированных символа

Область допустимых значений

STOP_SYM < 80₁₆ - иначе кодировка совпадет с символом из ISO-8859-5

80₁₆ <= СИМВ1, СИМВ2, ... <= FF₁₆ иначе не соответствует кодировке ISO-8859-5

WORD_BEG = 5C0₁₆ - по условию

WORD_BEG <= CURR < 79E₁₆

CUT = 00FF₁₆ исходя из цели задания

Расположение элементов в памяти

1. Команды: 3A3 - 3B5
2. Переменные: 39E - 3A2
3. Закодированные символы: 5C0 - ...

Трассировка

ЯСТРЕБ, первые 2 символа

Выполняемая команда		Содержимое регистров процессора после выполнения команды								Ячейка, содержимое которой изменилось после выполнения команды	
Адрес	Код	IP	CR	AR	DR	SP	BR	AC	NZVC	Адрес	Новый код
3A3	0200	3A4	0200	3A3	0200	000	3A3	0000	0100		
3A4	1203	3A5	1203	3A4	1203	000	03A4	0040	0100		
3A5	2F40	3A6	2F40	3A5	0040	000	0040	0040	0000		
3A6	F0FD	3A7	F0FD	3A6	F0FD	000	03A6	0040	0000		
3A7	AAF8	3A8	AAF8	5C0	C1CF	000	FFF8	C1CF	1000		
3A8	EEF8	3A9	EEF8	3A1	C1CF	000	FFF8	C1CF	1000	3A1	C1CF
3A9	2EF8	3AA	2EF8	3A2	00FF	000	FFF8	00CF	0000		
3AA	F00A	3AB	F00A	3AA	F00A	000	03AA	00CF	0000		
3AB	1302	3AC	1302	3AB	1302	000	03AB	00CF	0000		
3AC	1203	3AD	1203	3AC	1203	000	03AC	0040	0000		
3AD	2F40	3AE	2F40	3AD	0040	000	0040	0040	0000		
3AE	F0FD	3AF	F0FD	3AE	F0FD	000	03AE	0040	0000		
3AF	AEF1	3B0	AEF1	3A1	C1CF	000	FFF1	C1CF	1000		
3B0	0680	3B1	0680	3B0	0680	000	03B0	CFC1	1000		
3B1	2EF0	3B2	2EF0	3A2	00FF	000	FFF0	00C1	0000		
3B2	F002	3B3	F002	3B2	F002	000	03B2	00C1	0000		
3B3	00C1	3B4	1302	3B3	1302	000	03B3	00C1	0000		
3B4	CEEE	3A3	CEEE	3B4	03A3	000	FFEE	00C1	0000		

Доп. Задание

С ВУ-8 (клавиатура) вводится строка, enter - завершение ввода. После окончания ввода, на ВУ-5 (принтер) вывести столько символов тире, сколько букв во введенном слове. Далее пользователь нажимает любой символ на клавиатуре, затем enter, после чего программа подставляет букву в слово (перерисовывает вывод на принтер), если она там есть, так пока пользователь не угадает всё слово

Программа

closed_symbol: word 0x2E

word_length: word 0x00

word_index: word 0x65C

guessing_index: word 0x35C

saved_symbol: word ?

len_copy: word ?

not_guessed: word 0x00

start: cla

read: in 0x19

and #0x40

beq read

in 0x18

cmp #0x0A

beq next_line

st (word_index)+

ld closed_symbol

st (guessing_index)+

out 0xC

ld word_length

inc

st word_length

jump read

next_line: ld #0x0A

out 0xC

change_index: ld word_index

sub word_length

st word_index

ld guessing_index

sub word_length

st guessing_index

ld word_length

st len_copy

ld #0x00

st not_guessed

read_guessing: in 0x19

and #0x40

beq read_guessing

in 0x18

cmp #0x0A

beq open

st saved_symbol

jump read_guessing

open: ld (word_index)+

cmp saved_symbol

beq change_guessing

cmp (guessing_index)+

back: loop len_copy

jump open

jump change_guessing_index

change_guessing: st (guessing_index)+

jump back

change_guessing_index: ld guessing_index

sub word_length

st guessing_index

ld word_length

st len_copy

print_guessing: ld (guessing_index)+

out 0xC

cmp closed_symbol

beq plus

point: loop len_copy

```
jump print_guessing
```

```
ld not_guessed
```

```
cmp #0x00
```

```
beq exit
```

```
jump next_line
```

```
plus: ld not_guessed
```

```
inc
```

```
st not_guessed
```

```
jump point
```

```
exit: hlt
```

Вывод

В ходе выполнения лабораторной работы я научился осуществлять асинхронный обмен данными с внешними устройствами и попрактиковался в написании программ для БЭВМ на ассемблере.